

LAPORAN PERTEMUAN
PRAKTIKUM STRUKTUR DATA DAN ALGORITMA
PERTEMUAN 4
IMPLEMENTASI STACK DAN QUEUE BERBASIS LINKED LIST



Disusun Oleh:

Nama : Atalariq Barra Hadinugraha
NIM : 25/557554/SV/26192
Kelas : PSDA - B2
Dosen Pengampu : Dr. John Doe

D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

A. Tujuan Praktikum	5
B. Dasar Teori	5
1. Abstract Data Type (ADT)	5
2. Stack dan Queue	5
C. Hasil dan Pembahasan	5
1. Implementasi Stack	5
2. Implementasi Queue	6
3. Catatan Highlighting Bahasa Lain	6
4. Analisis Kompleksitas	6
D. Kesimpulan	6
Lampiran	7
Daftar Pustaka	8

Daftar Gambar

Gambar 1	Ilustrasi placeholder untuk diagram Stack vs Queue	5
----------	--	---

Daftar Tabel

Tabel 1	Kompleksitas Waktu Stack dan Queue Berbasis Linked List	6
---------	---	---

A. Tujuan Praktikum

1. Memahami konsep Abstract Data Type (ADT) Stack dan Queue beserta karakteristik urutannya masing-masing.
2. Mengimplementasikan struktur data Stack dan Queue menggunakan Linked List dalam bahasa pemrograman Python.
3. Menganalisis kompleksitas waktu operasi inti Stack dan Queue, meliputi Push, Pop, Enqueue, dan Dequeue.

B. Dasar Teori

1. Abstract Data Type (ADT)

Abstract Data Type (ADT) adalah model matematis untuk suatu tipe data yang didefinisikan melalui perilakunya dari sudut pandang pengguna, bukan melalui detail implementasinya [1]. ADT memisahkan antara **antarmuka** (operasi apa yang tersedia) dengan **implementasi** (bagaimana operasi itu dijalankan di memori).

2. Stack dan Queue

Stack adalah ADT yang mengikuti disiplin akses *Last In, First Out* (LIFO) [1]. Queue adalah ADT yang mengikuti disiplin akses *First In, First Out* (FIFO) [2].



Gambar 1: Ilustrasi placeholder untuk diagram Stack vs Queue

C. Hasil dan Pembahasan

1. Implementasi Stack

Kelas Stack diimplementasikan dengan sebuah list Python sebagai penyimpanan internal.

```
python
1  class Stack:
2      def __init__(self):
3          self._items = []
4
5      def push(self, value):
6          self._items.append(value)
7
8      def pop(self):
9          > Melempar IndexError alih-alih diam-diam mengembalikan None.
10         if not self._items:
11             raise IndexError("pop from empty stack")
12         return self._items.pop()
13
14     def peek(self):
15         return self._items[-1]
16
17     def is_empty(self):
18         return len(self._items) == 0
```

Kode 1: Implementasi kelas Stack

2. Implementasi Queue

python

```
1 from collections import deque
2
3
4 class Queue:
5     def __init__(self):
6         self._items = deque()
7
8     def enqueue(self, value):
9         self._items.append(value)
10
11    def dequeue(self):
12        if not self._items:
13            raise IndexError("dequeue from empty queue")
14        return self._items.popleft()
15
16    def is_empty(self):
17        return len(self._items) == 0
18
```

Kode 2: Implementasi kelas Queue berbasis collections.deque

Berikut demonstrasi eksekusi Stack dan output-nya berdampingan:

python

```
Eksekusi
1 s = Stack()
2 s.push(10)
3 s.push(20)
4 print(s.pop())
```

Output

20

3. Catatan Highlighting Bahasa Lain

Cuplikan PHP tanpa tag pembuka <?php dan cuplikan Blade tetap diwarnai, karena codeblock/code-figure mengalihkan highlighter-nya secara diam-diam (label tab tetap menampilkan nama bahasa aslinya):

php

```
1 public function edit(Project $project)
2 {
3     return view('projects.edit', ['project' => $project]);
4 }
```

Kode 3: PHP tanpa <?php.

blade

```
1 @if (session('success'))
2     <div class="alert">{{ session('success') }}</div>
3 @endif
```

Kode 4: Blade, diwarnai sebagai HTML.

4. Analisis Kompleksitas

Tabel 1: Kompleksitas Waktu Stack dan Queue Berbasis Linked List

Operasi	Stack	Queue
Push / Enqueue	$O(1)$	$O(1)$
Pop / Dequeue	$O(1)$	$O(1)$
Ruang (Space)	$O(n)$	$O(n)$

D. Kesimpulan

1. Stack sebagai ADT LIFO berhasil diimplementasikan.
2. Queue sebagai ADT FIFO berhasil diimplementasikan menggunakan collections.deque.
3. Kedua struktur data memberikan kompleksitas waktu $O(1)$ untuk operasi utamanya.

Lampiran

Kode sumber lengkap kelas Stack ada di bawah ini.

python

```
stack.py
1  class Stack:
2      def __init__(self):
3          self._items = []
4
5      def push(self, value):
6          self._items.append(value)
7
8      def pop(self):
9          if not self._items:
10             raise IndexError("pop from empty stack")
11             return self._items.pop()
12
13      def peek(self):
14          return self._items[-1]
15
16      def is_empty(self):
17          return len(self._items) == 0
18
```

Daftar Pustaka

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, 4th ed. The MIT Press, 2022.
- [2] R. Sedgewick dan K. Wayne, *Algorithms*, 4th ed. Addison-Wesley Professional, 2011.